Questionário investigativo do projeto integrador Full Stack

Identificação

Disciplina: Laboratório de Programação Full Stack

Data: ____/____/_____ Turma: _____ Grupo: _____

Integrante	Matrícula
1.	
2.	
3.	
4.	

Regra de resposta

Cada resposta deve apresentar: **ação executada**, **evidência coletada**, **arquivo e função localizados** e **explicação da relação entre o código e o resultado**. Respostas apenas conceituais não atendem à atividade.

Parte 1 Inicialização e arquitetura

1. Inicie a aplicação e registre a mensagem do terminal que prova que o servidor está disponível. Localize no comando executado o módulo e o objeto carregados pelo Uvicorn.

Ação executada: _____

Evidência: _____

Arquivo e função: _____

Explicação: _____

_____

2. Abra a interface e o Swagger. Registre os dois endereços e explique por que ambos pertencem à mesma aplicação.

Ação executada: _____

Evidência: _____

Arquivo e função: _____

Explicação: _____

_____

3. Localize o `lifespan`. Que instrução ocorre antes do `yield`? Mostre no código onde a tabela é criada e explique por que reiniciar não apaga os registros.

Ação executada: _____

Evidência: _____

Arquivo e função: _____

Explicação: _____

_____

4. Desenhe o fluxo de uma criação desde o clique em Salvar até a gravação no SQLite. Para cada etapa, informe o arquivo correspondente.

Ação executada: _____

Evidência: _____

Arquivo e função: _____

Explicação: _____

_____

Parte 2 Contrato da API

5. No Swagger, identifique método, URL, corpo JSON, resposta e código HTTP da criação. Relacione cada item ao conceito de contrato da API.

Ação executada: _____

Evidência: _____

Arquivo e função: _____

Explicação: _____

_____

6. Crie uma reserva válida pela interface. Registre a requisição no painel Rede do navegador ou repita a operação no Swagger. Localize a chamada `fetch`, a rota FastAPI e o modelo Pydantic envolvidos.

Ação executada: _____

Evidência: _____

Arquivo e função: _____

Explicação: _____

_____

7. Explique por que a API devolve `201` e não `200` na criação. Localize a configuração desse código no backend e o teste que o verifica.

Ação executada: _____

Evidência: _____

Arquivo e função: _____

Explicação: _____

_____

8. Envie `quantidade_pessoas` igual a zero pelo Swagger. Registre o `422`, identifique a informação de erro e localize a restrição correspondente no modelo.

Ação executada: _____

Evidência: _____

Arquivo e função: _____

Explicação: _____

_____

9. Consulte um identificador inexistente. Registre o `404`, localize a decisão no endpoint e explique por que essa situação pertence à API, embora a busca ocorra no serviço.

Ação executada: _____

Evidência: _____

Arquivo e função: _____

Explicação: _____

_____

Parte 3 Persistência e regras

10. Crie uma reserva, encerre o servidor, reinicie e consulte novamente. Registre a evidência de permanência e localize a instrução SQL responsável pela gravação.

Ação executada: _____

Evidência: _____

Arquivo e função: _____

Explicação: _____

_____

11. Cadastre duas reservas com o mesmo laboratório, data e turno. Registre a primeira resposta e a segunda. Localize a verificação do conflito, a conversão da exceção em `409` e o teste automatizado correspondente.

Ação executada: _____

Evidência: _____

Arquivo e função: _____

Explicação: _____

_____

12. Altere apenas o responsável de uma reserva. Registre o `PUT`, o JSON devolvido e as instruções do frontend, endpoint e serviço que participam do fluxo.

Ação executada: _____

Evidência: _____

Arquivo e função: _____

Explicação: _____

_____

13. Exclua uma reserva e tente consultá-la. Registre `204` e depois `404`. Explique por que uma resposta `204` não possui corpo e localize o tratamento desse caso no JavaScript.

Ação executada: _____

Evidência: _____

Arquivo e função: _____

Explicação: _____

_____

14. Use o filtro de turno. Registre a URL com query string, o resultado exibido e a montagem dinâmica da consulta SQL.

Ação executada: _____

Evidência: _____

Arquivo e função: _____

Explicação: _____

_____

15. Abra o resumo antes e depois de criar uma reserva noturna. Registre a mudança e localize a função que calcula `total` e `por_turno`.

Ação executada: _____

Evidência: _____

Arquivo e função: _____

Explicação: _____

_____

Parte 4 Testes automatizados

16. Execute `python -m pytest -v`. Registre o total de testes, o tempo e os estados apresentados. Explique como o pytest descobriu os arquivos e as funções.

Ação executada: _____

Evidência: _____

Arquivo e função: _____

Explicação: _____

_____

17. Escolha o teste do fluxo CRUD. Separe no código os trechos Arrange, Act e Assert, mesmo que não estejam comentados com esses nomes.

Ação executada: _____

Evidência: _____

Arquivo e função: _____

Explicação: _____

_____

18. Explique por que `TestClient` não exige Uvicorn. Indique o caminho percorrido por uma chamada `client.post` até a resposta testada.

Ação executada: _____

Evidência: _____

Arquivo e função: _____

Explicação: _____

_____

19. Localize `tmp_path`, `monkeypatch` e `with TestClient(app) as cliente`. Explique o papel de cada elemento e apresente uma evidência de que o banco normal não foi alterado pela suíte.

Ação executada	
Evidência	
Arquivo e função	
Explicação	

20. Modifique temporariamente uma asserção correta para provocar uma falha. Registre a mensagem do pytest, identifique valor obtido e esperado e depois restaure o teste.

Ação executada: _____

Evidência: _____

Arquivo e função: _____

Explicação: _____

_____

21. Execute a cobertura. Informe o percentual total, escolha duas linhas não cobertas e proponha cenários de teste capazes de executá-las.

Ação executada: _____

Evidência: _____

Arquivo e função: _____

Explicação: _____

_____

22. Compare o teste `test_rejeitar_conflito` com `test_servico_impede_conflito`. Explique a diferença entre testar pela API e testar diretamente a regra de negócio.

Ação executada: _____

Evidência: _____

Arquivo e função: _____

Explicação: _____

_____

Parte 5 Síntese do fluxo

23. Complete a cadeia com nomes reais do projeto: evento da interface → função JavaScript → método e rota HTTP → função do endpoint → modelo → função do serviço → SQL → código HTTP → atualização da interface.

Ação executada: _____

Evidência: _____

Arquivo e função: _____

Explicação: _____

_____

24. Escolha um defeito possível. Informe em qual camada ele surgiria, qual evidência o revelaria e qual teste deveria impedir sua volta.

Ação executada: _____

Evidência: _____

Arquivo e função: _____

Explicação: _____

_____

Entrega

- Questionário preenchido.
- Capturas numeradas E01 a E12 conforme o roteiro de evidências.
- Arquivo de texto com comandos executados.
- Indicação de arquivo, função e trecho de código em todas as questões que solicitam localização.

Roteiro de evidências

As evidências devem mostrar somente a área necessária para comprovar o resultado. Cada imagem deve conter uma legenda curta informando ação, resultado e questão relacionada.

Código	Evidência mínima	Questões
E01	Terminal com Uvicorn iniciado	1
E02	Interface e endereço no navegador	2
E03	Swagger com as rotas disponíveis	2 e 5
E04	Criação válida com resposta 201 e JSON	6 e 7
E05	Validação rejeitada com 422	8
E06	Consulta inexistente com 404	9
E07	Registro existente após reiniciar	10
E08	Duas tentativas e resposta 409	11
E09	Atualização por PUT	12
E10	Exclusão 204 e consulta 404	13
E11	Filtro e resumo alterado	14 e 15
E12	pytest e relatório de cobertura	16 a 22

Registro de confronto entre execução e código

Para cada evidência, informe:

1. O que foi executado.

2. O que foi observado.

3. Qual arquivo e função explicam o resultado.

4. Qual trecho confirma a explicação.

5. Qual teste protege esse comportamento ou qual teste deveria ser criado.